

COVERAGES

IN SYSTEM VERILOG

CONTENTS

1.coverage

2.types of coverages

3.code coverage

4.functional coverage

5.covergroup

6.coverpoint

7.bins

8.cross coverage

9.functional coverage methods

8.example codes

What is coverage?

coverage is a **verification metric** used to measure how much of the design functionality has been exercised by the testbench.

Types of Coverage in SystemVerilog:

- 1.Code coverage
- 2.Functional coverage

Code Coverage

- handled by simulator, not written manually
- Tells how much of the RTL code has been executed during simulation.

Types of code coverage:

1. **Line/Statement Coverage** → Are all lines executed?
2. **Branch Coverage** → Are all if/else paths taken?
3. **Condition Coverage** → Are all Boolean conditions evaluated to both TRUE and FALSE?
4. **Expression Coverage** → Are all possible results of expressions observed?
5. **Toggle Coverage** → how many times signal and ports are toggled during simulation time?
6. **FSM Coverage** → Are all states and transitions of FSM visited?

Functional Coverage

- written by verification engineer using covergroup, coverpoint, cross coverage.
- Measures whether all required **functional scenarios** are tested.
- Defined manually in testbench, independent of DUT implementation.

Uses:

Covergroup – container for coverage

Coverpoint – specifies which variable/signal values to track

Bins – categories of values to be counted

Cross coverage – combinations of multiple variables

1.Covergroup

- A container that holds coverage definitions.
- Declared inside a class or module.
- Can be sampled at specific events (like @(posedge clk)).

Syntax:

```
covergroup cg @(posedge clk);
```

```
coverpoint data;
```

```
endgroup
```

2.Coverpoint

- It specifies which **variable, signal, or expression** should be monitored for coverage.
- A coverpoint collects values of a variable.
- It divides those values into **bins** (categories) to track how many times each range/pattern has occurred.

Syntax:

```
covergroup cg @(posedge clk);  
coverpoint data; // auto-bins created by simulator  
endgroup
```

3.Bins in Functional Coverage

- **Bins** are **buckets/categories** that group values of a coverpoint.
- Simulator counts how many times each bin is hit during simulation.
- Used to measure whether required values/ranges have been tested.

Types of Bins:

1. Automatic Bins

- Created automatically when you don't define bins explicitly.
- Simulator divides the value space into bins (usually one bin per value if range is small).

Syntax:

```
coverpoint data; // automatic bins created
```

Example: if data is 2-bit, bins = {0,1,2,3}.

2. Explicit Bins

- When you define **bins manually**, they are called **explicit bins**.
- Explicit bins can be classified into **Scalar bins** and **Vector bins**.

1. Scalar Bins

- Define coverage for **single values or ranges**.
- No [] after the bin name.

Syntax:

```
coverpoint data {  
    bins zero  = {0};    // scalar bin for a single value  
    bins low   = {[1:5]}; // scalar bin for a range }
```

Scalar bins = one bin per definition.

2. Vector Bins

- Define coverage for a **set of multiple bins at once**.
- Use [] after the bin name.
- Simulator automatically creates one bin for each element in the range/list.

Syntax:

```
coverpoint data {  
    bins all_vals[] = {[0:15]}; // creates 16 bins, one for each value 0–15  
}
```

Vector bins = multiple bins created automatically for each value in range.

3. Transition Bins

- Track sequences of values (value changes).

Syntax:

```
coverpoint state {  
    bins idle_to_busy = (0 ==> 1);  
    bins busy_to_idle = (1 ==> 0);  
}
```

4. Wildcard Bins

- Allow don't-care bits (x or ?) in bin definitions.

Syntax:

```
coverpoint addr[3:0] {  
    wildcard bins upper_half = {4'b1???}; // covers 8–15  
}
```

5. Ignore Bins

- Values that you **don't care about** (excluded from coverage %).

Syntax:

```
coverpoint data {  
    ignore_bins unused = {0}; // not counted in coverage  
}
```

6. Illegal Bins

- Values that should **never occur**.
- If sampled, simulator throws an error.

Syntax:

```
coverpoint data {  
    illegal_bins invalid = {16,17,18};  
}
```

4. Cross Coverage

- Measures **combinations** of multiple coverpoints.
- While a **coverpoint** checks values of one variable, a **cross** checks **multi-variable combinations**.

Syntax:

```
covergroup cg @(posedge clk);  
    coverpoint opcode;  
    coverpoint size;  
    cross opcode, size;  
endgroup
```

- This checks not only individual opcodes and sizes, but also whether **all pairs of (opcode, size)** were tested.

Functional Coverage Methods

1. Creating / Sampling Coverage

sample()

- Manually triggers coverage sampling of a covergroup.
- Useful when you don't want event-based sampling.

Syntax:

```
covergroup cg;  
  coverpoint data;  
endgroup  
cg c1 = new();  
initial begin  
  data = 5;  
  c1.sample(); // manually sample coverage  
end
```

2.get_coverage()

- Returns overall coverage % of a covergroup (0–100).

Syntax:

```
$display("Coverage = %0.2f%%", c1.get_coverage());
```

3.get_inst_coverage()

- Returns coverage of a specific instance of the covergroup.

Syntax:

```
$display("Instance coverage = %0.2f%%", c1.get_inst_coverage());
```

4.set_inst_name("name")

- Assigns a custom name to a covergroup instance for reports.

Syntax:

```
c1.set_inst_name("MyDataCoverage");
```

5.option.goal

- Defines target coverage percentage (default = 100%).

Syntax:

```
covergroup cg;
```

```
    option.goal = 90; // stop collecting after reaching 90%
```

```
endgroup
```

6. start()

- Enables coverage collection.
- Coverage bins start counting when this is called.

7. stop()

- Disables coverage collection.
- After calling stop(), even if sample() or the event triggers, coverage will not be updated.

Use with sample()

- ❑ If you are using sample() manually, you can control *when* coverage is allowed to record using start() and stop().

Example:

initial begin

```
c1.start();
```

```
data = 5; c1.sample(); // recorded
```

```
c1.stop();
```

```
data = 10; c1.sample(); // not recorded
```

end

Example code using Coverpoints and Manual Sampling

```
module fn_cgrp;
  logic [3:0] data; // 4-bit variable
  logic [2:0] addr; // 3-bit variable
  logic en;        // 1-bit variable

  covergroup cg_group;
    option.per_instance = 1;
    cg1: coverpoint data; // auto bins 0–15
    cg2: coverpoint addr; // auto bins 0–7
    cg3: coverpoint en;   // auto bins 0–1
  endgroup
  cg_group cg = new();
  initial begin
    repeat(5) begin
      data = $urandom_range(0, 15);
      addr = $urandom_range(0, 7);
      en = $urandom_range(0, 1);
      cg.sample();
      $display("data=%0d , addr=%0d , en=%0d", data, addr, en);
    end
    $display("Coverage = %0.2f%%", cg.get_coverage());
  end
endmodule
```

Output:

```
data=3 , addr=2 , en=0
data=13 , addr=0 , en=0
data=0 , addr=4 , en=0
data=12 , addr=6 , en=1
data=8 , addr=1 , en=1
Coverage = 64.58%
```

Code using Explicit Scalar and Vector Bins

```
module fn_cgrp;
  logic [3:0]data;
  logic [2:0]addr;
  logic en;
  covergroup cg_group;
    option.per_instance=1;
    cg1:coverpoint data {bins b1  ={{5:$}};
                        bins b2[] ={2,3,6};
                        bins b3  =default;}
    cg2:coverpoint addr {bins b1  ={1,10,12};
                        bins b2[] ={{2:9},11};
                        bins b3[4] ={{0:7}};}
    cg3:coverpoint en {bins one={1}};
  endgroup
  cg_group cg=new();
  initial begin
    repeat(5) begin
      data=$random;
      addr=$random;
      en=$random;
      cg.sample();
      $display("data=%0d ,addr=%0d,en=%0d",data,addr,en);
    end
    $display("Coverage = %0.2f%%", cg.get_coverage());
  end
endmodule
```

Output:

```
data=4 ,addr=1,en=1
data=3 ,addr=5,en=1
data=5 ,addr=2,en=1
data=13 ,addr=6,en=1
data=13 ,addr=4,en=1
Coverage = 77.27%
```

Coverage with Event-Based Sampling (posedge clk)

```
module fn_cgrp;
  logic [3:0]data;
  logic [2:0]addr;
  logic clk;
  initial clk=0;
  always #10 clk=~clk;
  covergroup cg_group @(posedge clk);
    option.per_instance=1;
    cg1:coverpoint data;
    cg2:coverpoint addr;
  endgroup
  cg_group cg=new();
  initial begin
    repeat(5)
      begin
        @(posedge clk);
        data=$random;
        addr=$random;
        $display("data=%0d ,addr=%0d",data,addr);
      end
    $display("Coverage = %0.2f%%", cg.get_coverage());
    $finish;
  end
endmodule
```

Output:

```
data=4 ,addr=1
data=9 ,addr=3
data=13 ,addr=5
data=5 ,addr=2
data=1 ,addr=5
Coverage = 37.50%
```

Thank you!

